

系统开发工具基础·实验报告（第 13–16 题）

姓名：李云龙 学号：25030021043

2026 年 9 月 14 日

姓名	李云龙
学号	25030021043
实验日期	2026 年 9 月 14 日
实验环境	Windows 11 + WSL2 (Ubuntu 26.04 LTS); Python 3.14 + ruff 0.16 + pytest 9.1 + GNU Make 4.4.1 + jq 1.8.1

说明：本报告中各题“运行结果”均为在 WSL 中实际运行的输出；第 16 题的“失败门禁”输出与 wheel 的 SHA-256 均如实记录。

1 实验目的

1. 建立一个可执行的本地质量门禁（格式检查、静态检查、测试三合一），并理解“不全局忽略规则”的意义。
2. 理解 Make 的增量构建判据（目标与依赖的时间戳），写出依赖精确的 Makefile。
3. 掌握用 curl + jq 把本地 API 的 JSON 数据转换为可读 Markdown 报告。
4. 在陌生仓库中完成一次综合检查：用门禁暴露缺陷、定位修复、产出可校验的构建产物。

2 实验十三：建立可执行的本地质量门禁（q13）

2.1 最小配置（pyproject.toml）

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25030021043"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.ruff]
line-length = 88
```

```
src = ["src", "tests"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

只保留最小配置，没有使用任何全局忽略规则（如 `ignore = [...]`）。

2.2 测试（正常姓名 + 空白姓名，均有有效断言）

```
import sys

import pytest

from greetlab.cli import main

def test_greet_normal_name(monkeypatch, capsys):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "25030021043"])
    main()
    captured = capsys.readouterr()
    assert captured.out == "Hello, 25030021043!\n"

@pytest.mark.parametrize("blank", [" ", " ", "\t"])
def test_blank_name_exits_with_code_2(monkeypatch, capsys, blank):
    monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", blank])
    with pytest.raises(SystemExit) as excinfo:
        main()
    assert excinfo.value.code == 2
    assert capsys.readouterr().out == ""
```

2.3 质量门禁脚本（check.sh）

```
#!/usr/bin/env bash
# 本地质量门禁：格式检查 -> 静态检查 -> 测试，任一环节失败即返回非 0
set -euo pipefail
cd "$(dirname "$0")"

echo "== 1/3 ruff format --check =="
ruff format --check .

echo "== 2/3 ruff check =="
ruff check .

echo "== 3/3 pytest =="
python3 -m pytest

echo "全部通过 (exit 0) "
```

2.4 运行结果

```
$ ruff format .
4 files left unchanged

$ ./check.sh
== 1/3 ruff format --check ==
4 files already formatted
== 2/3 ruff check ==
All checks passed!
== 3/3 pytest ==
tests/test_cli.py .... [100%]
4 passed in 0.26s
全部通过 (exit 0)
```

2.5 要点分析

- 门禁把“格式 + 静态检查 + 测试”固化为一条可执行命令，本地开发与 CI 使用同一标准，避免“我这儿能过”；
- `set -euo pipefail` 保证任一环节失败立即中止并返回非 0，门禁才有“门”的作用；
- 不全局忽略规则是关键要求：一旦写 `ignore = [...]`，等于把规则悄悄关掉，门禁形同虚设；
- 两个测试各自带有效断言：正常路径断言输出内容，异常路径断言退出码并额外确认“没有输出问候语”。

3 实验十四：让 Make 只重建真正受影响的产物（q14）

3.1 源文件（题目给定）

data.csv:

```
name,value
alpha,2
beta,3
gamma,5
```

stats.py:

```
import csv

with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))
```

build_report.py:

```
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

report.md:

```
# Data Report
```

3.2 Makefile

```
.PHONY: all clean

all: stats.txt report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md stats.txt build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
```

3.3 验证结果

操作	实际现象
首次 make	依次执行 stats.py、build_report.py; stats.txt 内容为 10, report.txt 为 Data Report + Total: 10
无改动再 make	make: Nothing to be done for 'all'. (不执行任何配方)
touch data.csv 后 make	两个配方都执行 (data.csv 变新 -> stats.txt 重建 -> 进而 report.txt 重建)
只 touch report.md 后 make	只执行 build_report.py, 不牵连上游
make clean	只删除 stats.txt、report.txt; 源文件与 Makefile 均保留

3.4 要点分析

- Make 的重建判据是“目标文件与依赖文件的时间戳”：任一依赖比目标新，则执行配方；
- 因此依赖必须列全且精确：stats.txt 依赖 data.csv 与 stats.py，report.txt 依赖 report.md、stats.txt 与 build_report.py。漏列会漏建（改脚本却不重建），多列会白建（无关改动触发重建）；
- all、clean 不是磁盘上的真实文件，必须声明 .PHONY，否则一旦目录里出现同名文件，Make 会使用时间戳判定“已是最新”而跳过配方。

4 实验十五：把本地 API 数据转换为可读报告 (q15)

4.1 数据与脚本

packages.json（题目给定，5 条记录）保存于 q15 目录，用 `python3 -m http.server 8000` 提供本地 HTTP 访问。api_report.sh:

```
#!/usr/bin/env bash
# 从本地 HTTP 服务拉取 packages.json, 用 jq 筛选排序, 生成 Markdown 报告 summary.md
set -euo pipefail
cd "$(dirname "$0")"

url="{1:-http://127.0.0.1:8000/packages.json}"
tmp_json="{mktemp}"
trap 'rm -f "$tmp_json"' EXIT

if ! curl -fsS "$url" -o "$tmp_json"; then
    echo "错误: 无法从 $url 获取数据 (请确认已在 q15 目录启动 python3 -m http.server 8000) " >&2
    exit 1
fi

{
    echo "# Package Summary"
    echo
    echo "数据来源: ${url}; 筛选条件: status=active 且 downloads >= 100; 排序: downloads 降序, name 升序。"
    echo
    jq -r '["| name | version | downloads |", "| --- | --- | --- |"]
        + ([.[] | select(.status == "active" and .downloads >= 100)]
            | sort_by(-.downloads, .name)
            | map("| \(.name) | \(.version) | \(.downloads) |"))
        | .[]' "$tmp_json"
} > summary.md

cat summary.md
```

4.2 运行结果 (summary.md)

```
# Package Summary

数据来源: http://127.0.0.1:8000/packages.json; 筛选条件: status=active 且 downloads >= 100; 排序: downloads 降序, name 升序。

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

服务端日志确认请求真实发生:

```
127.0.0.1 - - [14/Sep/2026 14:01:22] "GET /packages.json HTTP/1.1" 200 -
```

4.3 要点分析

- 筛选结果正确: beta (status=inactive) 与 epsilon (downloads=80) 被排除;

- 排序正确：delta 与 gamma 同为 450，按 name 升序时 delta 在前；alpha 120 最后；
- `curl -fsS` 的组合意义：`-f` 让 HTTP 4xx/5xx 也返回非零退出码（配合 `set -e` 及时中止）、`-s` 静默进度条、`-s` 保留错误信息；
- jq 用一条表达式完成“筛选 + 多键排序”（`select + sort_by(-.downloads, .name)`），比用 shell 循环处理 JSON 更可靠。

5 实验十六：修复并交付一个陌生的小型工具仓库（q16）

5.1 Makefile

```
.PHONY: check build clean

check:
    ./check.sh

build:
    python3 -m build

clean:
    rm -rf build dist src/*.egg-info
```

5.2 综合检查过程

第 1 步：基线检查

```
$ make check
4 passed in 0.23s
全部通过 (exit 0)
```

第 2 步：把问候语临时改成字面量（模拟“坏实现”）

```
print("Hello, name!")    # 无论传入什么姓名，都输出固定的字面量
```

第 3 步：门禁成功拦住

```
$ make check
>      assert captured.out == "Hello, 25030021043!\n"
E      AssertionError: assert 'Hello, name!\n' == 'Hello, 25030021043!\n'
FAILED tests/test_cli.py::test_greet_normal_name
1 failed, 3 passed in 0.21s
make: *** [Makefile:4: check] Error 1
```

第 4 步：定位并修复（恢复 f-string，使用真实参数）

```
print(f"Hello, {a.name}!")
```

第 5 步：复验与构建

```
$ make check
4 passed in 0.17s
全部通过 (exit 0)

$ make build
Successfully built greetlab_25030021043-0.1.0.tar.gz and greetlab_25030021043-0.1.0-py3-none-any.whl
```

第 6 步：计算 wheel 的 SHA-256

```
09f86db07ed755d4dbb0e73f8746527a843a7950c22504969812ed05335ff8e3 dist/greetlab_25030021043-0.1.0-py3-none-any.whl
```

第 7 步：提交——最终改动以内容聚焦的 Git 提交入库（提交信息中记录了 wheel 的 SHA-256 前 8 位）。

5.3 要点分析

- 一个“陌生仓库”能不能放心修改，取决于**是否存在可执行的质量门禁**：测试失败直接导致 `make check` 失败（退出码非 0），说明门禁真的在起作用；
- 修复遵循最小改动原则（只恢复被改坏的一行），修改后用**同一条命令**复验，避免引入新的偏差；
- “可交付”的标志是产物可校验：生成 wheel 并给出 SHA-256，接收方可用同一命令重建并比对哈希。

6 遇到的问题与解决

问题	原因	解决
<code>make / jq</code> 命令不存在	Ubuntu 基础镜像未预装	<code>apt install make jq</code>
测试导入不到 <code>src</code> 布局的包	<code>src</code> 布局下包不在 <code>sys.path</code>	增加 <code>confest.py</code> 把 <code>src</code> 加入 <code>sys.path</code>
启动本地服务后又“消失”	<code>pkill -f http.server</code> 的正则匹配到执行该命令的 <code>shell</code> 自身, 把自己杀掉	改为脚本内用 <code>!</code> 记录 PID + <code>trap</code> 收尾 (<code>demo.sh</code>)
坏实现是否会被发现	需要可执行门禁验证	故意改成字面量后 <code>make check</code> 立即失败, 证明门禁有效

7 实验总结

通过本次实验，我建立了由 `ruff format`、`ruff check` 与 `pytest` 组成的本地质量门禁，并体会到“门禁必须可执行、且不能靠全局忽略规则绕过”；理解了 Make 依据时间戳做增量构建的原理，写出依赖精确、`all/clean` 均为 `.PHONY` 的 Makefile，并验证了“只改下游输入则只重建下游产物”；掌握了用 `curl -fsS` 与 `jq` 把本地 HTTP 服务上的 JSON 转换为带筛选、多键排序的 Markdown 报

告；最后在陌生仓库中完成综合检查——先用门禁暴露缺陷、最小修复、再复验并产出带 SHA-256 的 wheel，完整体验了可交付的工程闭环。

附录：文件清单

w4/README.md	第 13--16 题说明（步骤、原理、实测数据）
w4/q13/check.sh	第 13 题质量门禁脚本
w4/q13/pyproject.toml	第 13 题 ruff / pytest 最小配置
w4/q13/tests/test_cli.py	第 13 题测试（正常姓名 + 空白姓名）
w4/q13/src/greetlab/	第 13 题包源码
w4/q14/Makefile	第 14 题构建规则（依赖精确 + .PHONY）
w4/q14/{data.csv,stats.py,build_report.py,report.md}	第 14 题源文件
w4/q14/{stats.txt,report.txt}	第 14 题生成的产物
w4/q15/packages.json	第 15 题数据
w4/q15/api_report.sh	第 15 题报告生成脚本
w4/q15/demo.sh	第 15 题一键演示（起服务 + 生成 + 收尾）
w4/q15/summary.md	第 15 题生成的 Markdown 报告
w4/q16/Makefile	第 16 题 check / build / clean
w4/q16/check.sh	第 16 题质量门禁（同 q13）
w4/q16/src/greetlab/cli.py	第 16 题修复后的实现
w4/q16/dist/	第 16 题构建产物（wheel, SHA-256 见 5.2）
实验报告.md / 实验报告.tex / 实验报告.pdf	本报告